



## 30+ Wireshark Filters for **Threat Detection**

| No | Threat Type                | How It Works                                                      | Display Filter                                                                                                                                                                                                                                                                                                                                                                                                                         | Detection Method                                                                                                            |
|----|----------------------------|-------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| 1  | HTTP GET Flooding          | High volume of HTTP GET requests to overload server               | <code>http.request.method == "GET"</code>                                                                                                                                                                                                                                                                                                                                                                                              | Check for abnormal number of requests from same source IP; investigate suspicious patterns in user-agent strings            |
| 2  | Suspicious HTTP User-Agent | Irregular User-Agent strings used to mask identity                | <code>http.user_agent</code>                                                                                                                                                                                                                                                                                                                                                                                                           | Examine HTTP headers for unusual or malformed User-Agent strings; correlate requests with known malicious IP addresses      |
| 3  | Malicious HTTP User-Agent  | Specific malicious user-agent strings used by attackers           | <code>http.user_agent contains "malicious_string"</code>                                                                                                                                                                                                                                                                                                                                                                               | Check for known malicious user-agent strings; correlate with other suspicious activities                                    |
| 4  | SQL Injection via HTTP     | Malicious SQL commands inserted into application queries via HTTP | <code>http.request.full_uri contains "SELECT" or http.request.full_uri contains "UNION" or http.request.full_uri contains "INSERT" or http.request.full_uri contains "DELETE" or http.request.full_uri contains "OR" or http.request.full_uri contains "AND" or http.request.full_uri contains "'" or http.request.full_uri contains "--" or http.request.full_uri contains "/" or http.request.full_uri contains "xp_cmdshell"</code> | Inspect HTTP GET and POST requests for SQL keywords in unexpected places; analyze request parameters for special characters |
| 5  | Cross-Site Scripting (XSS) | Malicious scripts injected into web pages viewed by other users   | <code>http.request.uri contains "&lt;script&gt;" or http.request.uri contains "%3Cscript%3E" or http.request.uri</code>                                                                                                                                                                                                                                                                                                                | Examine HTTP request URIs and POST data for script tags or encoded JavaScript                                               |

|    |                               |                                                                    |                                                                                                                     |                                                                                                                                  |
|----|-------------------------------|--------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
|    |                               |                                                                    | contains "javascript:" or<br>http.request.uri contains "onload=" or http.request.uri contains "onerror="            |                                                                                                                                  |
| 6  | HTTP Directory Traversal      | Attempts to access restricted directories via manipulated URLs     | http.request.uri contains ".." or<br>http.request.uri contains "%2e%2e%2f" or http.request.uri contains "%2e%2e%5c" | Analyze HTTP request URIs for directory traversal patterns like "../"; look for encoded versions                                 |
| 7  | Suspicious HTTP POST Requests | HTTP POST requests used for data exfiltration or malicious uploads | http.request.method == "POST"                                                                                       | Review HTTP headers for POST requests with large payloads or to unknown IP addresses; inspect content for encoded data           |
| 8  | DNS Tunneling                 | DNS queries manipulated to transmit data, bypassing firewalls      | dns                                                                                                                 | Inspect DNS query field for abnormal payload sizes or repeated requests; analyze DNS response time for unusual delays            |
| 9  | DNS Tunneling (Advanced)      | Long DNS query names used for data exfiltration                    | dns.qry.name.len > 50                                                                                               | Look for unusually long DNS query names; analyze frequency and patterns of queries                                               |
| 10 | DNS Amplification             | Small request triggers large DNS responses, overwhelming target    | dns.flags.response == 1 and<br>dns.resp.len > 500                                                                   | Check DNS response section for large-sized responses and unexpected source addresses; look at TTL field for unusually low values |
| 11 | DNS Poisoning                 | DNS responses altered to redirect users to malicious websites      | dns.flags.rcode != 0 or dns.resp.ttl < 60                                                                           | Analyze DNS response headers for mismatched IP addresses or altered TTL values; check response time field for unexpected delays  |

|    |                                  |                                                                       |                                                              |                                                                                                                                              |
|----|----------------------------------|-----------------------------------------------------------------------|--------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| 12 | SMB Brute Force                  | Multiple login attempts via SMB protocol to guess valid credentials   | <code>smb.cmd == (=0x73)</code>                              | Check SMB headers for failed login attempts; monitor SMB command response times                                                              |
| 13 | SMB Brute Force (Specific)       | Failed SMB login attempts                                             | <code>smb.cmd == 0x73 and smb.nt.status == 0xc000006d</code> | Monitor for repeated failed SMB authentication attempts from the same source                                                                 |
| 14 | SMB Null Session Exploitation    | Attempts to access SMB shares without authentication                  | <code>smb.sessetup and smb.account == ""</code>              | Monitor SMB setup requests for null session attempts; check for empty native OS fields in SMB negotiation                                    |
| 15 | SYN Flood Attack                 | Flood target server with SYN packets without completing TCP handshake | <code>tcp.flags.syn == 1 and tcp.flags.ack == 0</code>       | Review TCP headers for high volume of SYN packets without corresponding ACKs; look for delayed ACKs or connection resets                     |
| 16 | ICMP Flooding (Ping of Death)    | Numerous ICMP requests to overload device                             | <code>icmp.type == 8 and ip.len &gt; 65535</code>            | Review ICMP headers for high frequency of Echo Request packets from single source; inspect for ICMP packets larger than standard 64 bytes    |
| 17 | ICMP Tunneling                   | Data exfiltration or covert channel using ICMP packets                | <code>icmp.type == 8 and ip.payload_len &gt; 64</code>       | Detect ICMP echo requests with unusually large payloads                                                                                      |
| 18 | ARP Spoofing                     | Falsified ARP messages link wrong MAC addresses with IP addresses     | <code>arp.duplicate-address-frame</code>                     | Inspect ARP header for mismatches between IP and MAC addresses; check for ARP requests with identical source IPs but different MAC addresses |
| 19 | ARP Spoofing (Duplicate Address) | Conflicting ARP messages to manipulate network traffic                | <code>arp.duplicate-address-detected</code>                  | Identify instances of duplicate IP addresses in ARP traffic                                                                                  |

|    |                                  |                                                                                     |                                                                                                                                  |                                                                                                                                                  |
|----|----------------------------------|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| 20 | FTP Plaintext Authentication     | FTP transmits login credentials in plaintext                                        | <code>ftp.request.command == "USER" or ftp.request.command == "PASS"</code>                                                      | Review FTP request headers for visible usernames and passwords; inspect packet payloads for plaintext data in FTP stream                         |
| 21 | FTP Command Injection            | Attacker attempts to inject malicious commands into FTP sessions                    | <code>ftp.request.command contains ";" or ftp.request.command contains "</code>                                                  | Look for FTP commands containing semicolons or other potential command separators                                                                |
| 22 | SSL/TLS Heartbleed Vulnerability | Exploit in OpenSSL library allowing memory content theft                            | <code>ssl.heartbeat.payload_length &gt; 16384</code>                                                                             | Check SSL/TLS heartbeat messages for abnormally large payload lengths; analyze response data for potential memory leaks                          |
| 23 | SSL Downgrade Attack             | Force use of weaker SSL/TLS encryption protocols                                    | <code>ssl.record.version &lt; 0x0303</code>                                                                                      | Check SSL/TLS handshake headers for negotiation with older protocol versions; inspect certificate chains for self-signed or expired certificates |
| 24 | Suspicious TLS Certificate       | Invalid or self-signed TLS certificates used to compromise encrypted communications | <code>ssl.handshake.type == 11 and (x509sat.OID == 2.5.4.3 and x509ce.keyIdentifier.id == x509ce.subjectKeyIdentifier.id)</code> | Review SSL handshake header for certificates signed by unknown authorities; verify encryption protocol for downgrade attempts                    |
| 25 | DHCP Starvation                  | Attacker floods network with DHCP requests to exhaust IP pool                       | <code>dhcp.option.dhcp == 1</code>                                                                                               | Monitor for high volume of DHCP Discover messages from multiple MAC addresses; check for rapid succession of requests from single source         |
| 26 | Rogue DHCP Server                | Unauthorized DHCP server assigns IP addresses to redirect traffic                   | <code>dhcp.option.dhcp == 2</code>                                                                                               | Review DHCP Offer packets and compare server IP address to authorized DHCP servers; check for abnormal lease durations or renewals               |

|    |                                  |                                                                  |                               |                                                                                                                                           |
|----|----------------------------------|------------------------------------------------------------------|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| 27 | SNMP Community String Bruteforce | Attempts to guess SNMP community strings for unauthorized access | <code>snmp.community</code>   | Monitor for multiple SNMP requests with different community strings from same source; check for high rate of SNMP authentication failures |
| 28 | RDP Bruteforce                   | Repeated login attempts to RDP services                          | <code>tcp.port == 3389</code> | Analyze TCP traffic on RDP port for multiple failed connection attempts; look for patterns in timing and source of connection requests    |
| 29 | SSH Bruteforce                   | Multiple SSH login attempts to guess valid credentials           | <code>tcp.port == 22</code>   | Monitor SSH traffic for repeated failed authentication attempts; analyze timing and source of connection requests                         |
| 30 | VOIP Call Hijacking              | Unauthorized interception or manipulation of VOIP calls          | <code>sip</code>              | Examine SIP traffic for unexpected INVITE or BYE messages; look for discrepancies in call IDs or sudden changes in RTP endpoints          |